

CANNBot智能体，畅享算子开发新旅程

<https://gitcode.com/cann/cannbot-skills>

分享嘉宾：譙竣仁
2026.06.27 中国上海



目录

Part 1 CANNBot概述

Part 2 环境搭建

Part 3 核心 workflow

Part 4 实操演示

CANNBot是什么？



核心定位

CANNBot是面向 **CANN 开发** 的系列智能体，旨在大幅降低算子开发难度，提升开发效率。本仓库为其提供可复用的 **Skills 模块**，目前已实现 **Ascend C / PyPTO / TileLang / Triton / CATLASS 算子开发全流程覆盖**，未来将拓展至 CANN更多技术领域。



技术愿景

CANNBot 的目标是成为 **CANN 社区** AI 辅助研发的**基础设施**，持续提升异构计算生态的开发效率和工程质量。



核心价值

- ✓ 标准化开发工作流，简化算子开发
- ✓ 自动设计方案、编写代码、审查质量
- ✓ 输出包含 Kernel 和 Tiling 的可编译代码



目标用户

- 👤 CANN社区入门开发者
- 👤 AscendC/PyPTO/TileLang/Triton/ CATLASS 算子开发者
- 👤 昇腾NPU平台AI应用开发者
- 👤 希望贡献 Skills / Agents 的社区贡献者

70+

个Skills

覆盖算子开发全流程

7

个阶段

开发周期从天级缩短到小时级

3

层架构

Teams→Agents→Skills

CANNBot 架构图

Harness

六层约束体系

L0 数学 L1 硬件
L2 反模式 L3 数值
L4 性能 L5 流程

四层反馈网络

L1 微循环（分钟）
L2 功能对拍(十分钟)
L3 性能优化（小时级）
L4 知识进化（天级）

两级熵减机制

迭代层（算子内·分钟级）
知识层（跨算子·天级）

成熟度模型

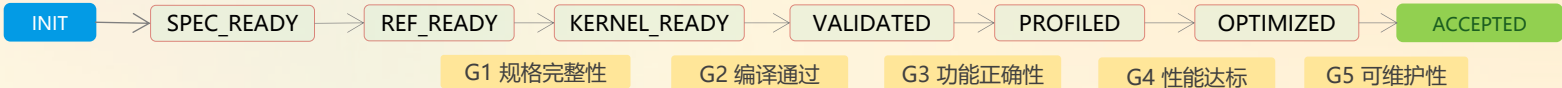
L1基础约束
→ L2闭环验证
→ L3性能闭环
→ L4知识驱动
→ L5自适应进化

用户需求

(数学表达式 · shape/dtype · 语义 · 硬件平台 · 性能目标)

编排器

(状态机驱动 · 合约检查 · 自动回退 · 超时降级)



SubAgent

Designer

TestDesigner

Developer

Evaluator

Profiler

Archivist

Skills

Arch Skill

Kernel Skill

Tiling Skill

Perf Skill

Review Skill

.....

基础大模型层 — GLM / Qwen / Opus / DeepSeek / GPT

基础设施和工具

编译工具链

测试框架

Profiling

Memory

数据飞轮

.....

最终交付物

设计文档

测试用例

算子文件

精度报告

性能报告

知识沉淀

增训

训练数据采集

正样本 · 修复样本 · 偏好样本 · 拒答样本 · 反模式样本

后训练路线

S1: SFT 监督微调
S2: DPO 偏好优化
S3: Reward Model

分Agent独立训练

Spec → 规格生成能力
Kernel → 首版正确率
Perf → 优化建议质量
Ref → 测试生成质量

上限门禁

编译通过率 ≥ 当前版本
功能通过率提升 ≥ 5%
修复轮次下降 ≥ 15%
灰度上线 → 指标达标
→ 正式切换

CANNBot当前70+ Skills, 覆盖算子开发全流程, 持续丰富扩展中

Ascend C 算子开发 (28+)

ascendc-api-best-practices API最佳实践
ascendc-blaze-best-practice
Matmul/Cube/GEMM/BMM 单算子直调生成
ascendc-code-review 代码检视
ascendc-precision-debug 精度调试与症状速查
ascendc-crash-debug 算子卡死/崩溃调试
ascendc-direct-invoke-template Ascend C
Kernel 直调工程
ascendc-direct-invoke-to-registry-invoke
Ascend C Kernel 直调转 msOpGen 自定义算子

Ascend C 算子开发

ascendc-docs-gen 算子文档生成
ascendc-docs-search 算子文档搜索
ascendc-env-check 环境检查
ascendc-npu-arch NPU架构知识与芯片映射
ascendc-performance-best-practices 性能优化实践
ascendc-precision-debug 精度调试与症状速查
ascendc-regbase-best-practice RegBase 开发
ascendc-registry-invoke-template 自定义算子模板
ascendc-registry-invoke-to-direct-invoke 自定义算
子转 <<<>>> kernel 直调
ascendc-runtime-debug 运行时错误码解析

Ascend C 算子开发

ascendc-st-design 测试设计
ascendc-task-focus 任务聚焦
ascendc-tiling-design Tiling设计方法论
ascendc-ut-develop 单元测试开发
ascendc-whitebox-design 白盒测试用例
aiss-tiling-solver 求解最优Giling参数
cann-env-setup CANN开发环境配置
ops-precision-standard 算子精度标准
ops-profiling 算子性能采集分析
ops-simulator NPU仿真器
ops-spec-gen 生成或校验算子 spec.yaml

PyPTO 算子开发 (8个)

pypto-api-explore API可行性探索分析
pypto-golden-generate Golden参考实现生成
pypto-intent-understand 算子需求意图图理
pypto-op-design 算子方案设计与模板
pypto-op-develop 算子代码实现与测试
pypto-op-perf-tune 算子性能分析与调优
pypto-precision-compare 算子精度调试
pypto-precision-debug 算子精度问题调试

TileLang 算子开发 (5个)

tilelang-api-best-practices API最佳实践
tilelang-op-design 算子设计
tilelang-op-develop 算子开发
tilelang-programming-model 编程模型指南
tilelang-review 代码检视

Triton 算子开发 (5个)

triton-latency-optimizer 代码性能逐步优化
triton-op-coding Triton内核代码生成
triton-op-designer 算法设计指导
triton-op-verifier 精度与性能验证
triton-task-extractor 代码检视任务提取与构建

CANNBot当前70+Skills, 覆盖算子开发全流程, 持续丰富扩展中

NPU 模型推理优化 (11个)

model-infer-fusion 融合算子替换
model-infer-graph-mode torch.compile图模式
model-infer-kvcache KVCache+FA优化
model-infer-migrator 框架适配与基线
model-infer-multi-stream
model-infer-parallel-impl
model-infer-parallel-analysis 并行策略分析
model-infer-precision-debug 推理精度诊断
model-infer-prefetch Prefetch 预取优化技能
model-infer-runtime-debug 推理运行时诊断与修复
model-infer-superkernel SuperKernel适配

CATLASS 算子开发 (3个)

catlass-op-desgin CATLASS内核设计
catlass-op-develop CATLASS内核代码开发
catlass-op-perf-tune CATLASS性能调优

Torch 入图开发 (6个)

torch-custom-ops-guide 自定义算子入图
torch-npugraph-ex-compile-error-diagnosis
npugraph_ex 编译期报错诊断
torch-npugraph-ex-dfx-triage npugraph_ex DFX
分诊
torch-npugraph-ex-knowledge npugraph_ex知识
torch-npugraph-ex-runtime-error-diagnosis
npugraph_ex 运行时错误诊断
torch-npugraph-ex-template npugraph_ex 模式
MRE 模板

其他算子工具 (2个)

cuda2ascend-simt CUDA算子到昇腾的SIMT迁移
ops-easyasc-dsl easyasc DSL → AscendC

基础设施 - GitCode自动化协作工具 (4个)

gitcode-issue-gen PR关联Issue生成
gitcode-issue-handler Issue端到端处置
gitcode-pr-handler PR标题描述自动生成
gitcode-toolkit API/Git工具集

目录

Part 1 CANNBot概述

Part 2 快速上手

Part 3 核心 workflow

Part 4 实操演示

环境要求与前置条件

硬件要求

昇腾NPU设备

Ascend 910及后续系列产品

内存

建议16GB以上

存储

建议50GB可用空间

软件要求

CANN版本

8.0.RC2及以上

操作系统

EulerOS 2.10 / CentOS 7.6 / Ubuntu 18.04+

CLI工具

OpenCode、Claude Code、TRAE等

环境检查清单



NPU设备就绪

npu-smi info 能正常输出



CANN已安装

版本 ≥ 8.0.RC2



CLI工具就绪

OpenCode、Claude Code等

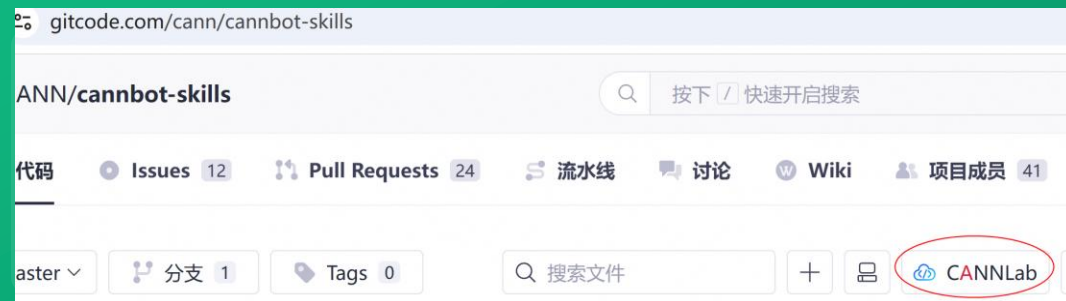


网络连接

能访问GitCode仓库

云开发环境(推荐)

CANN社区云开发环境支持一键启动，NPU设备、CANN环境、OpenCode工具等环境依赖均已默认安装。



快速上手



CLI交互式安装工具(推荐)

交互式命令安装，快速上手，轻松配置，[适合多项目开发场景](#)

```
# 1. 安装cannbot交互式安装工具, 根据指导, 一键安装
npx @cannbot-ai/install-helper

# 1. 一键获取所有CANNBot skills
npx skills add https://gitcode.com/cann/cannbot-skills.git --list
```



启动CLI工具

在初始化完成的目录下执行以下命令启动交互界面：

```
# 启动OpenCode CLI
opencode

# 启动后，输入算子开发需求
帮我开发一个abs算子，支持float16...
```



校验检查项



Skills目录存在

.opencode/skills/ 目录不为空



Agents目录存在

.opencode/agents/ 目录不为空



配置文件正确

AGENTS.md 和 opencode.json 存在



CLI能正常启动

opencode 命令无报错



常见问题

如何更新Skills?

重新执行skills获取命令

安装失败怎么办?

检查CANN版本和网络连接

目录

Part 1 CANNBot概述

Part 2 环境搭建

Part 3 自动开发 workflow

Part 4 实操演示

Ascend C: 七阶段自动化 workflows



质量控制

每个阶段完成后才能进入下一阶段，确保开发质量

迭代优化

阶段4-5支持循环迭代，持续优化算子性能

Agent驱动

每个阶段由专门Agent执行，自动完成各项任务

工作流触发：所有算子开发任务会自动加载 `ascendc-kernel-develop-workflow` 技能，按阶段执行。详见AGENTS.md配置文件。

阶段1-2：环境检查与需求分析

1

环境检查

确保开发环境就绪



NPU设备查询

检查NPU是否可用，获取设备信息



CANN环境验证

检查CANN版本、环境变量配置



依赖检查

验证必要的库和工具是否安装

自动执行环境检查

`ascendc-env-check skill`

2

需求分析

理解算子功能和要求



算子功能理解

分析算子的数学表达式和计算逻辑



数据类型确定

支持的数据类型(float16/float32等)



Shape分析

输入输出Tensor的Shape范围



性能要求

吞吐量、延迟等性能指标



Agent自动化: CANNBot的 **architect** Agent会自动分析需求，调用 `ascendc-npu-arch` 和 `ascendc-api-best-practices` Skills，生成设计方案。

阶段3：算子实现 - Tiling设计



什么是Tiling?

Tiling是将大规模数据计算任务 **切分成小块**，分配到多个AI Core并行执行的技术。合理的Tiling策略能显著提升算子性能。

Tiling设计方法论

1 分析数据特征

理解输入数据的Shape、数据类型、访问模式

2 确定切分策略

选择合适的切分维度(N/H/W/C等)

3 计算Tiling参数

block_dim、tile_size等参数计算

4 Buffer规划

合理分配GM、UB、L1内存

 **自动Tiling设计：**CANNBot根据算子类型自动选择合适的Tiling方案：Element-wise算子按数据块均匀切分，Reduce算子按Reduce轴切分，MatMul算子采用分块矩阵乘法策略。调用 `ascendc-tiling-design Skill` 自动生成。

Tiling数据结构

```
// Tiling参数示例
struct AddTiling {
    uint32_t blockDim; // 核数
    uint32_t tileSize; // 块大小
    uint32_t totalSize; // 总大小
    uint32_t splitAxis; // 切分轴
};
```

切分策略

多核切分

将数据分配到多个AI Core

UB切分

适配Unified Buffer容量

阶段3：算子实现 - Kernel开发



Kernel代码生成

CANNBot自动生成符合 **Ascend C规范** 的Kernel代码，包括数据搬入、计算、数据搬出三个核心步骤。

代码生成流程

1

分析需求

理解算子功能和数据特征

2

选择模板

基于算子类型选择代码模板

3

生成代码

填充模板生成完整Kernel代码

4

优化调整

应用最佳实践和性能优化

代码质量保证



遵循API最佳实践



符合NPU架构特性



自动内存对齐



流水线优化



developer Agent自动完成

产出物结构

```
ops/{operator}/
├── CMakeLists.txt
├── {operator}_kernel.asc
├── build.sh
├── run.sh
├── scripts/
├── docs/
│   ├── DESIGN.md
│   └── PLAN.md
└── tests/
```

Kernel函数结构

```
// Kernel函数结构
extern "C" __global__ __aicore__ void
abs_kernel(...) {
    // 1. 数据从GM搬入UB
    DataCopy(xUB, xGM, size);
    // 2. 执行计算(Abs)
    Abs(zUB, xUB, size);
    // 3. 数据从UB搬出到GM
    DataCopy(zGM, zUB, size);
}
```

阶段4-5：验证优化与问题处理

4

验证与优化

确保算子正确性和性能

编译算子

使用build.sh编译生成算子二进制文件

运行测试

执行run.sh验证算子功能正确性

性能分析

使用profiling工具分析性能瓶颈

```
# 编译和测试
bash build.sh
bash run.sh
```

 **迭代优化：**阶段4-5支持循环迭代，持续优化直到满足要求

5

问题处理

调试和修复问题

精度调试

对比CPU和NPU输出，定位精度问题

Skill: ascendc-precision-debug

运行时调试

解析错误码，排查Kernel挂起等问题

Skill: ascendc-runtime-debug

自动修复

CANNBot自动诊断并提供修复建议

 **智能诊断：**Agent自动分析问题并提供解决方案

阶段6-7：性能验收与总结

6

性能验收

采集性能数据并判定达标状态

性能数据采集

调用msprof op采集8个CSV指标文件

性能达标判定

对照性能标准判定是否达标

瓶颈定位与优化

定位瓶颈类型并给出优化建议

最终判定

PASS

PASS WITH NOTES

FAIL

7

完成总结

汇总开发结果并生成文档

总分与代码路径

性能概要

Task Duration、主导流水、达标状态

关键问题列表

文档归档

DESIGN.md / PLAN.md / REVIEW.md / WALKTHROUGH.md

目录

Part 1 CANNBot概述

Part 2 环境搭建

Part 3 核心 workflow

Part 4 实操演示

CANNBot使用示例视频

CANNBot 能力规划地图

Spec Agent
需求 → 结构化规格

shape/dtype/layout
/
/reduce语义/动态
shape/边界条件...
→ spec.yaml

— Skill —
ops-spec-
generation

Environment Agent
环境检查

NPU架构知识、芯片型号/ NPU设备检查/CANN环境验证

— Skill —
npu-arch
env-check (NPU设备检查, CANN环境验证)

Reference Agent
CPU 真值 + 测试集

CPU reference逻辑/
random/boundary/
extreme/
dynamic_shape
→
reference.py
test_cases.yaml

— Skill —
ops-reference-
and-test

Kernel Agent
NPU 算子代码生成

AscendC
SIMD/SIMT
PyPTO/TileLang/A
CLNN
→ kernel.asc

— Skill —
api-best-
practices
kernel-develop-
workflow
ascend-simd
ascend-simd
pypto
tilelangcustom-
op-to-kernel-
launch (自定义算
子转 <<<>>>
kernel 直调)

Performance Agent
性能分析与优化

Profiling瓶颈识别
/Tile/并行
/Vector/Cube/MC2/
FA/Conv/融合策略
→
optimization_plan.y
aml

— Skill —
ops-performance-
analysis
ops-precision-
debug
ops-runtime-
debug

Validation Agent
用例生成与自动测试验证

编译检查、
边界测试/动态shape
测试
→
optimization_plan.y
aml

— Skill —
ut-develop
st-develop
whitebox-design

Knowledge Agent
知识沉淀与复用

高性能模板·失败案例归档
反模式规则·Best Practice
模板检索·相似算子召回
→ 模板库

— Skill —
docs-search
docs-generation

现在，开始您的CANNBot之旅~ 让高效开发算子触手可及 ^_^

<https://gitcode.com/cann/cannbot-skills>

CANNBot代码仓库地址
欢迎star、fork、download、PR



Watch 17 Starred 1080 Fork 684

已存在的fork项目

Junren6415 / skills_7845

+ 创建fork

项目介绍

CANNBot 是面向 CANN 开发的
可复用的 Skills 模块。

邀请您扫码加入 “CANNBot Skills开源交流群”

群聊：CANNBot Skills开源交流
2群



群聊：CANNBot Skills开源交流
3群



该二维码7天内(6月17日前)有效，重新进入将更新

该二维码7天内(6月17日前)有效，重新进入将更新

注：“CANNBot Skills开源交流1群”，已超过200人，不能扫码，可通过好友邀请入群。

Thank you.



上CANN社区获取干货



关注CANN公众号获取资讯

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

